Machine Learning Report

Car License Plate Identification with a Jetson Nano

Author 1:	Mansour Ahmady Phoulady
Matriculation No.:	7025671
Course of Studies:	Business Intelligence and Data Analytics
Author 2:	Ranjeetsing Tawar
Matriculation No.:	7026116
Course of Studies:	Business Intelligence and Data Analytics
Author 3:	Hemanth Prabhakaran
Matriculation No.:	7026000
Course of Studies:	Business Intelligence and Data Analytics

First examiner:	Prof. Dr. Elmar Wings
Submission date:	November 6, 2024

University of Applied Sciences Emden/Leer · Faculty of Technology ·
Mechanical Engineering Department
Constantiaplatz 4 · 26723 Emden · http://www.hs-emden-leer.de

Contents

Contents i

List of Figures iii

List of Tables v

Acronyms vii

I. Introduction 1

1. Abstract 2

2. Introduction 3
2.1. Challenges 3
2.2. Proposed Solution 4
2.3. Report Structure 4

II. Domain Knowledge 7

III. KDD Process Structure 8

IV. Packages 9

3. Libraries/Packages List 10
3.1. Introduction 10

V. Conclusion 11

List of Figures

List of Tables

Acronyms

Part I.

Introduction

1. Abstract

Current techniques for license plate detection and recognition primarily rely on small, often non-global, and non-generic datasets due to the absence of publicly accessible large and diverse datasets. In this study, we introduce the Car License Plate Detection Dataset (CLPD), a substantial and comprehensive collection of license plate images. Each image is meticulously annotated with bounding box coordinates. To the best of our knowledge, CLPD stands as the most extensive publicly available license plate dataset to date, comprising more than 433 unique car images and the exclusive provision of vertex location annotations.Leveraging CLPD, we present an innovative Convolutional Neural Network (CNN) model capable of predicting bounding boxes and simultaneously recognizing associated license plate numbers swiftly and accurately.Through a series of close-range experiments, our model outperforms current object detection and recognition methods in terms of both precision and speed.

Keywords: License plate detection · Car License Plate Detection Dataset (CLPD) · Bounding box annotation · Vertex location annotations · Convolutional Neural Network (CNN) · Object detection

2. Introduction

In present day scenario the security and authentication is very much needed to make a safety world.Beside all security one vital issue is recognition of number plate from the car for authorization. In the busy world everything cannot be monitor by a human, so automatic license plate recognition is one of the best application for authorization without involvement of human power.
The suggested solution belongs to the category of challenges categorized as "image processing," which can be tackled using Convolutional Neural Networks (CNNs), a subtype of Artificial Neural Networks. CNNs are chiefly designed for intricate image-based pattern recognition tasks, owing to their precise yet uncomplicated architecture. This solution comprises two distinct phases to determine whether a vehicle may be granted parking privileges. In the initial stage, the system undergoes training and testing using a substantial dataset to identify various segments of the license plate and accurately extract the pertinent information. Following this training, the system is deployed onto the Jetson Nano. In the subsequent stage, the Jetson Nano's onboard camera captures the license plate information from the car's image. If the recognition process is successful, a green indicator illuminates, signifying a valid license plate recognition. Conversely, if recognition errors occur or if the identified license plate is not found in the database, a red indicator is activated to signal that parking is not allowed for the vehicle.[**Sandha:2017**]

2.1. Challenges

Recognizing vehicle plates has historically presented several challenges. These challenges include:[**Awalgaonkar:2021**]

- Variability in Plate Design: License plates come in a wide range of designs and formats, with variations in colors, fonts, and sizes. This variability can make it difficult to create a single, universal recognition system.

- Speed of Vehicles: In scenarios like toll booths or fast-moving traffic, the speed of vehicles can make it challenging to capture license plates quickly and accurately.

- Varying Plate Sizes: Different countries and regions have license plates of varying sizes, and this adds complexity to recognition systems that need to adapt to these differences.
- Foreign License Plates: In areas with international traffic, systems must be capable of recognizing a wide range of license plate designs from different countries.
- Processing Power: Efficiently processing images and running recognition algorithms in real-time, especially in scenarios with a large volume of vehicles, requires substantial computational power.

2.2. Proposed Solution

Recognizing vehicle plates presents various challenges, and YOLO (You Only Look Once) can assist in addressing some of these challenges[**Sung:2020**]. These challenges encompass:

- Variability in Plate Design: YOLO can aid in localizing and identifying license plates. However, the recognition of specific design elements, such as characters, colors, and fonts, may necessitate additional processing steps, such as Optical Character Recognition (OCR).
- Speed of Vehicles: YOLO is specifically designed for real-time object detection, making it well-suited for scenarios involving fast-moving vehicles. Nevertheless, the accuracy of license plate capture at high speeds is contingent on the camera setup and image quality.
- Varying Plate Sizes: YOLO exhibits adaptability to objects of varying sizes, including license plates, rendering it versatile for deployment in regions with different plate dimensions.
- Foreign License Plates: YOLO can be trained to recognize a diverse array of license plate designs, making it an apt choice for areas with international traffic.
- Processing Power: YOLO is renowned for its real-time processing capabilities. However, its actual performance is influenced by the hardware employed. High-performance GPUs are often necessary for achieving real-time processing.

2.3. Report Structure

1. **Introduction**: Provides an overview of the project or subject matter.

2. **Challenges**: Identifies and discusses the obstacles or difficulties faced in the project.
3. **Data Version Control**: Discusses methods and best practices for managing and versioning data.
4. **Data Description**: Describes the characteristics and attributes of the data used in the project.
5. **Hardware Description**: Provides information about the hardware infrastructure utilized in the project.
6. **Development Environment and Software Description**: Describes the software tools and environment used for development.
7. **Data Mining**: Discusses techniques and processes for extracting insights and patterns from data.
8. **KDD Process**: Explores the Knowledge Discovery in Databases (KDD) process for extracting useful information from data.
9. **Development Folder Structures**: Describes the organization and structure of folders and directories used in development.
10. **Document Developer**: Discusses the documentation requirements and practices for developers.
11. **Programming Concepts**: Covers fundamental concepts and principles of programming relevant to the project.
12. **Requirements.txt**: Specifies the dependencies and requirements for the project, typically in a text file format.
13. **Software Tests**: Discusses the testing strategies and methodologies employed in the project.
14. **Deployment**: Covers the process of deploying the software or solution into a production environment.
15. **Monitoring**: Describes the methods and tools used for monitoring the performance and behavior of the deployed system.
16. **Program Flowchart**: Provides a visual representation of the flow of the program or system logic.
17. **Bill of Materials**: Lists the components and resources required for the project, typically used in manufacturing contexts.
18. **Numpy Package**: Discusses the features and usage of the NumPy library for numerical computing in Python.

19. **Pandas Package**: Discusses the features and usage of the Pandas library for data manipulation and analysis in Python.

20. **Matplotlib Package**: Discusses the features and usage of the Matplotlib library for data visualization in Python.

21. **Conclusion**: Summarizes the findings, outcomes, and conclusions drawn from the project.

22. **Data Sheets**: Provides detailed documentation or specifications about the datasets used in the project.

Part II.

Domain Knowledge

Part III.

KDD Process Structure

Part IV.

Packages

3. Libraries/Packages List

3.1. Introduction

Data science has become an integral part of various industries, ranging from finance to healthcare, and has revolutionized the way businesses operate. In recent years, there has been an explosion in the amount of data generated, leading to a growing need for tools that can handle large and complex datasets efficiently. This has led to the development of a wide range of data science packages that provide functionality for data manipulation, visualization, and machine learning. These data science packages are essential tools for any data scientist, as they provide the necessary functionality to efficiently analyze and manipulate data, build powerful models, and gain insights from complex datasets. The packages discussed in this report - pandas, numpy, matplotlib, datetime, seaborn, keras, lightgbm, and tensorflow - are among the most popular and widely used data science packages. They are open-source, easy to use, and provide extensive documentation, making them accessible to both novice and experienced data scientists.

Part V.

Conclusion